Harpocrates — Steganography Framework: How It Works

1. Project Overview

Harpocrates is a research-grade steganography framework. It lets you hide encrypted secret data inside innocuous-looking cover media (images, audio, URLs, and — planned — video) and later extract it back, provided you know the password. The name comes from the Greek god of silence and confidentiality, which is exactly what the project is about: covert, protected communication.

The repository is organized to support an academic thesis structure:

- **`.venv/`** — local Python virtual environment (Python 3.9) with all dependencies installed.
- **`backend/`** — the actual working code: a `modules/` package containing every embedder and steganalysis routine, a FastAPI application scaffold (`app/` with `api/`, `core/`, `models/`, `services/` subpackages, currently empty), the dependency manifest `requirements.txt`, and a pytest test suite.
- **`frontend/`** — empty; a planned web/UI layer.
- **`docs/`** — empty; this document lives here.
- **`evaluation/`** — empty scaffold (`results/`, `test_corpus/`) for the evaluation chapter.
- **`references/`** — reference implementations studied to inform this design.

The design goal shared by every embedder is the classic steganography triad: **capacity** (how much you can hide), **imperceptibility** (how hard it is to notice), and **robustness** (how well the secret survives extraction). Every module implements the same interface, the same security layer, and the same metric reporting, so algorithms can be swapped through configuration — classical (LSB, S-UNIWARD, QIM) today, neural (VideoSeal) later.

2. The Core Foundations (shared across all modules)

All modules depend on shared infrastructure defined in `backend/modules/`.

2.1 Payload framing — `modules/base.py`

Before any secret bytes are hidden, they are wrapped in a **framing header** so the extractor knows exactly how many bits to read and can verify integrity. The header is 14 bytes, big-endian:

```
[MAGIC (4)][VERSION (1)][FLAGS (1)][LENGTH (4)][CRC32 (4)]
 b"HSTG"       version      bitfield     uint32      uint32
```

- **MAGIC** — the literal bytes `b"HSTG"`. The extractor checks this first; a mismatch means "wrong carrier / wrong key / nothing embedded".
- **VERSION** — header format version (currently 1).
- **FLAGS** — a bitfield. Bit 0 (`FLAG_ENCRYPTED = 0x01`) says the payload is AES-encrypted; bit 1 (`FLAG_COMPRESSED = 0x02`) is reserved for future compression.
- **LENGTH** — the number of payload bytes that follow the header.
- **CRC32** — a checksum of the original payload, used as a final integrity check: decrypt, recompute CRC, compare.

An embedded payload therefore always looks like `[header 14 bytes][payload bytes]`. The extractor reads the header, learns `LENGTH`, reads exactly that many payload bytes, decrypts, and validates the CRC. A wrong password makes AES-GCM authentication fail, producing a graceful error.

2.2 Standard result type — `StegoResult`

Every embed operation returns a standardized object:

- `stego_media` — the cover with hidden data (ndarray, bytes, or string),
- `metrics` — a `MetricsBundle` with quality numbers,
- `algorithm` and `domain` — labels (e.g. `"image_lsb"` / `"spatial"`),
- `meta` — free-form algorithm options.

The uniform shape means evaluation tooling can treat every algorithm identically.

2.3 The abstract interface — `BaseEmbedder`

`BaseEmbedder` is an ABC with three abstract methods that every algorithm implements identically:

- `embed(cover, payload, key)` → `StegoResult` — hide `payload` (bytes) inside `cover` with the given password (`key`).
- `extract(stego, key)` → `bytes` — recover and return the hidden payload.
- `capacity(cover)` → `int` — the maximum payload size in bytes, minus the header overhead.

Each subclass sets `name`, `domain`, and optional `requires_torch`. The explicit intent (per the docstring) is that classical and neural (VideoSeal) embedders can be swapped purely through configuration because they share this interface.

3. Security Layer — `crypto_utils.py`

Before hiding, payloads are encrypted with standard, audited primitives from the `cryptography` library.

3.1 Encryption: AES-256-GCM

- A fresh, random **salt (16 bytes)** and **nonce (12 bytes)** are generated per message.
- The user password is stretched into a **32-byte (256-bit) AES key** using **PBKDF2-HMAC-SHA256 with 100,000 iterations**.
- The payload is encrypted with **AES-GCM**, which produces ciphertext **plus a 16-byte authentication tag**. The tag means any tampering, or a wrong password, fails decryption loudly.
- The final blob is packed as `[salt][nonce][ciphertext + tag]`.

3.2 Decryption

Decryption reverses the process: split the blob, re-derive the same key from password + salt, decrypt, and let GCM verify the tag. If the password is wrong (or the carrier was corrupted), decryption raises `ValueError` — which is why the "wrong key" tests pass in every test suite.

3.3 Pseudo-random ordering — `generate_prng_seed`

Random-order embedders need to scatter bits across the cover deterministically so that both sender and receiver visit the same locations. The seed is computed as `SHA-256(password)`, whose first 8 bytes are interpreted as a little-endian 64-bit integer. That integer feeds numpy's `RandomState`, so the same password always produces the same scrambling with no side information.

Notes worth flagging for the thesis:

- Encryption guarantees **confidentiality and integrity** of the hidden message; key material never travels with the file.
- PBKDF2 with 100k iterations is reasonable but modest by 2026 standards; Argon2id would be a stronger KDF.
- The PRNG seed is a raw hash of the password — fine for ordering, but a salted derivation would be more conservative.

4. Metrics — `metrics.py`

Every embed computes an objective before/after comparison so imperceptibility claims can be quantified and logged for the evaluation chapter:

- **PSNR** (dB) — image/video quality: `20·log10(max / sqrt(MSE))`; higher is better (>40 dB is typically imperceptible for 8-bit images).
- **SSIM** — structural similarity in `[-1, 1]`; 1.0 means identical.
- **SNR** (dB) — signal-to-noise ratio for audio signals (used by the audio modules).
- **BER** — bit error rate between the original and extracted payload; 0.0 means lossless extraction.
- **BPP** — bits per pixel/sample, a normalized measure of embedding capacity.

`MetricsBundle` collects all of these and serializes to a flat dict for CSV/JSON logging by the evaluation harness.

5. Image Modules (`backend/modules/image_stego/`)

5.1 LSB baseline — `lsb.py`

The reference spatial-domain method, studied against OpenStego's Java LSB implementation.

Embedding flow:

1. Encrypt the payload with AES-GCM, compute its CRC, build the 14-byte header, and concatenate `header + encrypted`.
2. Compute how many bits fit: `H × W × 3 channels × bits_per_channel`. If the payload does not fit, `bits_per_channel` auto-increases from 1 up to 3.
3. Flatten the RGB image. The payload bit stream (`np.unpackbits`) is grouped into `bits_per_channel`-wide chunks, converted to integer "low bits" values, and written into the chosen values with `(value &amp; clear_mask) | new_low` — LSB-first per value.
4. Placement is either **sequential** (raster scan order) or **random** (a password-seeded permutation that the receiver re-creates deterministically).
5. PSNR/SSIM/BPP metrics are computed on the fly; `bits_per_channel` and `random_order` are recorded in `meta`.

Extraction flow: read only the bytes needed (header first → learn the length → read the full payload), trying bits-per-channel 1→3, checking the magic marker, then decrypting and CRC-validating. A wrong key produces `ValueError: Failed to extract…`.

The module also ships `embed_image_file` / `extract_image_file` helpers that round-trip through PIL and save the stego image as lossless PNG.

5.2 Adaptive (S-UNIWARD-inspired) — `adaptive.py`

Classical LSB treats all pixels equally, which is statistically detectable. This module instead embeds **preferentially in texture-rich regions** (edges), where changes are masked by visual noise — the core idea of S-UNIWARD by Holub & Fridrich (2012), simplified to Sobel gradients.

Cost map: for every pixel channel, horizontal and vertical Sobel magnitudes are computed; `cost = 1 / (gradient + 1e-4)`, raised to the power `alpha` (default 1.0; a higher alpha is more selective).

Key trick — determinism without side information: the cost is computed on the **LSB-masked image** (`pixel &amp; 0xFE`, bit 0 cleared). Because embedding only ever flips bit 0, the cover and stego produce the *identical* cost map — so the receiver recomputes the same embedding order with no side channel.

Location selection: a **stable** `argsort` of the flattened cost map picks the `n_bits` lowest-cost locations. The stable sort guarantees that equal-cost ties resolve identically at embed and extract time.

Embed: a vectorized LSB flip at the selected flat indices. **Extract:** recompute the cost map, select locations, read the header, then the exact payload length, then decrypt + CRC-validate.

The result, per the module's intent, is better PSNR/SSIM and much harder to detect with chi-square/RS attacks than uniform LSB.

6. Audio Modules (`backend/modules/audio_stego/`)

6.1 Time-domain LSB — `time_lsb.py`

The "lossless carrier" fallback: hide bits directly in the LSBs of **16-bit PCM sample values** (WAV/FLAC). It mirrors the image LSB logic on a 1-D sample stream:

- Sample values are viewed as unsigned 16-bit for clean bit manipulation.
- 1–4 bits per sample (`bits_per_sample`, default 1).
- Samples are permuted with the password-seeded PRNG (`random_order=True` by default).
- Metrics: **SNR** for quality and **BER = 0.0** by construction (a lossless carrier round-trips exactly).

Capacity in bytes is `(N_samples × bits_per_sample / 8) − header_size`. Extraction reads the header first to learn the exact length, reads that many bytes, decrypts, and CRC-validates.

6.2 Frequency-domain STFT-QIM — `stft_qim.py`

This is the most intricate audio module. Instead of hiding in raw samples, it hides in the **magnitude spectrum** of the signal using **Quantization Index Modulation (QIM)**.

Why non-overlapping blocks? The docstring explains a subtle pitfall. With an overlapping windowed STFT, modifying only magnitudes produces a spectrum that no real signal possesses — so ISTFT→re-STFT would not recover the modified magnitudes (empirically ~50% BER). With **non-overlapping blocks**, each block's FFT is self-consistent: `irfft` → `int16` → `rfft` recovers the magnitudes up to rounding noise. Empirically this yields BER = 0 at ~75 dB SNR for `delta >= 2e-3`.

The algorithm:

1. Split the signal into **non-overlapping frames** (default 1024 samples).
2. Normalize to `[-1, 1]`; for each frame compute a real FFT → magnitude + phase.
3. **QIM on magnitudes:** in the embedding band (default 4–16 kHz, outside the dominant low-frequency psychoacoustic energy) compute `q = round(mag / delta)` with a **fixed step** `delta` (default 4e-3 in normalized units, chosen above the int16 + FFT round-trip noise floor).
4. **Enforce parity:** if `q mod 2` does not match the desired bit, move `q` to the nearest integer with the right parity; then set `mag = q · delta`. A bit is encoded by whether the quantized magnitude lands on an odd or even step.
5. `irfft` each block, reassemble, clip to int16.

Extraction re-runs the block FFT, computes `round(mag / delta)`, and reads the parity. No password is needed for ordering, but decryption still requires the key. The embed step also performs a cheap self-check: it re-extracts the payload and records BER = 0.0 (or 1.0) so the logged BER is real rather than assumed.

Capacity: `number_of_frames × bins_in_band` bits. `bins_in_band` follows from the sample rate, frame size, and the 4–16 kHz band.

7. Link Modules (`backend/modules/link_stego/`)

Two text-steganography techniques hide data inside URLs and text, for channels where images or audio are not available.

7.1 URL query-parameter permutation — `link_stego.py`

Idea: a URL query with N parameters can be ordered in N! ways; each ordering encodes `log2(N!)` bits. The cover URL is canonicalized by **sorted parameter keys**, the payload (header + encrypted bytes) is converted to a single big integer, and that integer is mapped to one specific permutation using the **factorial number system** (Lehmer code):

- `_int_to_permutation(n, size)` greedily resolves the integer into a permutation using successive factorials.
- `_permutation_to_int(perm)` inverts it.

The receiver parses the query's current parameter order, converts that permutation back to an integer, and recovers the original bytes. A subtlety documented in the code: the framed payload always begins with `b"HSTG"` whose first byte `0x48` is nonzero, so recovering the *minimal* big-endian byte length of the integer faithfully reconstructs the original bytes. 6 parameters ≈ 9.5 bits; 8 parameters ≈ 15.3 bits — a low-capacity, high-plausibility channel.

7.2 Zero-width characters (ZWC) — `link_stego.py`

Uses four **invisible Unicode** code points: U+200B (Zero Width Space), U+200C (Zero Width Non-Joiner), U+200D (Zero Width Joiner), and U+FEFF (Zero Width No-Break Space). Each carries 2 bits (a base-4 choice), so a text's "injection points" — conservatively one per 3 visible characters — absorb ~2 bits each.

Embedding strips any existing ZWC characters, encodes the framed payload as 2-bit pairs mapped to the four characters, and weaves them after every 3rd visible character. Extraction is a regex for the four ZWC code points, a 2-bits-per-character decode, re-packing to bytes, header parsing, and decryption.

This survives copy-paste and most CMS pipelines — but not Unicode normalization or aggressive HTML/whitespace stripping, a caveat worth noting in the thesis.

8. Steganalysis (the detector side) — `attacks.py`

To prove the embedders "resist steganalysis," the framework includes the two classical statistical detectors cited in the literature, and the test suite uses them to demonstrate resilience.

8.1 Chi-square attack (Westfeld & Pfitzmann, 1999)

Sequential LSB embedding flips bits so that each **Pair of Values** `(2i, 2i+1)` becomes nearly equal in frequency. The detector computes the chi-square statistic comparing observed even counts against the pair mean, and the interpretation is inverted relative to a naive uniform test:

- clean image → pairs not equalized → large χ^2 → p ≈ 0;
- LSB stego → pairs equalized → small χ^2 → p ≈ 1.

Hence `stego_probability = 1 − CDF(χ², df)` and `detected = stego_probability > (1 − alpha)`.

8.2 RS analysis (Fridrich et al., 2001)

Forms sequential pixel groups of size 3, defines a smoothness (discrimination) function `f = Σ |adjacent differences|`, and measures how it changes under LSB flips applied with a positive mask and a negative mask. It counts **Regular** (R) and **Singular** (S) groups for each mask; the deviation `ΔR = |R_M − R_{−M}|` estimates the embedded payload via the closed form `p = (2 ± sqrt(4 − 16·ΔR)) / 8`, clipped to [0, 1]. `detected` is true when the estimated payload exceeds 5%.

8.3 `self_test_image`

A convenience wrapper that runs both detectors on a (cover, stego) pair and reports the chi-square stego-probability, the RS payload estimate for each, and a combined verdict — "DETECTED" if the stego-probability rose by more than 0.1 or the payload estimate rose by more than 0.05.

The embodied expectation (mirrored in the tests): **sequential LSB is detectable; random-order and adaptive (S-UNIWARD-like) hiding resists these classical detectors.**

9. Testing

`backend/tests/` is a pytest suite. From `backend/`, run it with `pytest tests/`. Coverage by file:

- **test_image_lsb.py** — sequential and random round-trips, wrong-key rejection, exact capacity math, automatic bit-depth bump for oversized payloads, oversize rejection, PSNR/SSIM thresholds, empty payload.
- **test_image_adaptive.py** — round-trip, wrong key, texture preference (edge pixels modified first), deterministic extraction, capacity, alpha parameter effect, keyless determinism.
- **test_audio.py** — time-domain LSB (random and sequential, stereo shape preserved, capacity, wrong key) and STFT-QIM (clean round-trip, capacity, wrong key).
- **test_link.py** — ZWC round-trip / invisibility / wrong key / capacity, URL-permutation round-trip, Lehmer code math, insufficient-parameters guard.
- **test_steganalysis.py** — chi-square result sanity and detection of heavy LSB, RS estimates on clean images, self-test structure, and the key claim: *adaptive resists better than sequential.*

10. Planned / Scaffolded Infrastructure

The application scaffold and repositories are prepared but not yet implemented:

- **FastAPI + Pydantic** app — `app/api`, `app/core`, `app/models`, `app/services` exist as empty packages, intended to wrap the embedders behind REST endpoints.
- **Celery + Redis + PostgreSQL + SQLAlchemy + Alembic** — async job queue, task metadata persistence, and migrations.
- **Frontend** (`frontend/`) — planned, empty.
- **Evaluation harness** (`evaluation/results`, `evaluation/test_corpus`) — empty; `modules/metrics.py` is the designed logging layer for it.
- **Video channel** — `modules/video_stego` is a stub today, but `requirements.txt` already pins `torch==2.1.2`, and the `references/videoseal` checkout shows the intended neural direction (embedder/extractor U-Nets with augmentation-based robustness, PixelSeal/VideoSeal lineage).

The `references/` tree documents the design lineage: **openstego** (Java LSB/DCT/DWT), **javid-steganography** (simple image LSB), **AlphaSteg** (another GUI approach), **GBRAS-Net** (deep-learning steganalysis with trained S-UNIWARD/WOW detectors and SRM kernels), and **videoseal** (state-of-the-art video watermarking) — the "classical → neural" evolution this project is tracing.

11. End-to-End Walkthrough

Here is what actually happens when you hide a message in an image:

1. You call `LSBEmbedder().embed(cover_image, b"Top secret", "my password")`.
2. `SteganoCrypto.encrypt_payload` derives a 256-bit key from password + random salt, encrypts the message with AES-GCM, and returns `[salt][nonce][ciphertext + tag]`.
3. A `PayloadHeader` is built with `length = len(encrypted)`, `flags = 0x01` (encrypted), and `crc32` of the original message. The 14-byte header is prepended to the ciphertext.
4. Capacity is checked; if the message is too large for 1 bit/channel, the embedder silently uses 2 or 3 bits per channel.
5. Every RGB byte of the image is flattened; a raster or password-seeded random order selects which low-bit slots receive the framed bytes, and bit-clear-and-OR writes them in. The result is pixel-indistinguishable to a human eye (PSNR typically > 40 dB).
6. Metrics are computed and returned alongside the stego image.

To retrieve the message later (with the password):

1. `extract()` reads the first 14 bytes of LSBs. If the first 4 bytes are not `b"HSTG"`, this is not our carrier; otherwise the header reveals the payload length.
2. Read exactly that many payload bytes; decrypt with AES-GCM using the salt and nonce stored in the ciphertext; GCM verifies the tag against the password.
3. CRC check passes ⇒ the original message is returned. A wrong password fails GCM authentication and raises `ValueError` — no data leaks.

The same skeleton — encrypt → frame → embed → metrics → header read-back → decrypt → CRC → return — applies to all five modules; only the hiding channel changes (pixel channel LSBs, texture-weighted LSBs, PCM LSBs, FFT magnitude parity, URL parameter order, or zero-width characters).

12. Strengths, Limitations, and Roadmap

Strengths

- Uniform interface and uniform metrics ⇒ benchmarkable, swappable algorithms.
- Real, audited cryptography (AES-256-GCM) plus integrity framing (CRC32) on every payload.
- Numpy-vectorized hot loops throughout; fast and memory-friendly.
- Deterministic adaptive extraction (LSB-masked cost map + stable sort) needs no side channel.

Limitations

- Sequential LSB is statistically detectable at high embedding rates — the project's own chi-square/RS detectors prove it.
- STFT-QIM has modest capacity, and its zero-BER guarantee relies on `delta` ≈ 4e-3 being calibrated above the int16 + FFT round-trip noise floor.
- ZWC carriers can be destroyed by Unicode normalization or whitespace stripping on hostile platforms.
- The URL-permutation channel has very low capacity (log2(N!) bits per URL).
- The deployed stack (FastAPI, Celery, DB, frontend, evaluation pipeline) is scaffolded, not yet implemented.

Roadmap (as implied by the references)

1. Implement the video channel with a VideoSeal-style neural embedder.
2. Wire all embedders into the FastAPI service with async job processing.
3. Build the evaluation corpus and run the planned PSNR/SSIM/SNR/BER benchmark across LSB, adaptive, QIM, URL, ZWC, and the neural model, under real steganalysis detectors.

Appendix — Quick File Map

Path	Purpose
`backend/modules/base.py`	`BaseEmbedder` interface, `PayloadHeader` framing, `StegoResult`
`backend/modules/crypto_utils.py`	AES-256-GCM encryption, PBKDF2 key derivation, PRNG seeding
`backend/modules/metrics.py`	PSNR, SSIM, SNR, BER, BPP, `MetricsBundle`
`backend/modules/image_stego/lsb.py`	sequential / random LSB image embedding (1–3 bits/channel)
`backend/modules/image_stego/adaptive.py`	Sobel-based S-UNIWARD-style adaptive embedding
`backend/modules/audio_stego/time_lsb.py`	PCM time-domain LSB audio embedding
`backend/modules/audio_stego/stft_qim.py`	block-rFFT magnitude QIM audio embedding
`backend/modules/link_stego/link_stego.py`	URL query permutation + zero-width character embedding
`backend/modules/steganalysis/attacks.py`	chi-square and RS-analysis detection
`backend/tests/`	pytest suites for all modules
`backend/app/`	FastAPI-format scaffold (work in progress)
`backend/requirements.txt`	pinned dependencies
`references/`	prior art: OpenStego, Javid, AlphaSteg, GBRAS-Net, VideoSeal